Based on the above statements, we can make a valid statement as follows:

```
x = n1; //
```

The above statement implies that `x` and `n1` are equivalent and will have the same properties. The only difference lies in the fact that the value of the pointer `x` can be changed, while the pointer `n1` will always point to the first of the 15 elements of type `int`.

Similarly, if we declare a statement `n1= x;`, the statement will be considered invalid as `n1` is an array and operates as a constant pointer. Since we cannot assign values to constants, the above statement will not work. Now let us consider another program based on pointers.

```
# Program 3: A program to show various expressions related to pointers
#include <iostream.h>

int main ()
{
   int n1[ 5] ;   // an array n1 declared of type int
   int * q;            // a pointer declared  of type int
   q = n1;
*q = 1;
   q++;
*q = 2;
   q = &n1[ 2] ;
*q = 3;
   q = n1+ 3;
*q = 4;
   q = n1;
*(q+4) = 5;
   for (int x=0; x<5; x++)
      cout << n1[ x] << ", ";
```